Lab 3 explained

G M Robi Farhan Rojob | 73997 | Machine Learning, Group 01

Use this as your personal study guide. The report, code, plots and measured results are in the existing course project's lab03 folder. You still need to write Question 3 calculations on paper and understand your conclusions.

What the lab is asking

Task	What to produce
1 - Data	Fashion-MNIST; 55,000 train, 5,000 validation, 10,000 test; exact transforms; batch 128.
2 - Models	The specified MLP, CNN and two-layer patch-attention classifier.
3 - Costs	torchinfo parameter counts, paper FLOPs, THOP comparison, and median inference timings.
4 - Training	Five epochs per model, Adam at 0.001, and official test accuracy.
5 - Trade-offs	Accuracy-versus-parameters and accuracy-versus-latency plots; book-based explanation.
6 - Evidence	report.md and a genuine Git commit. Add your paper scans before final submission.

The numbers in square brackets in the manual are minutes, not marks.

The size contradiction

The exact MLP has 235,146 parameters, CNN 20,490 and attention 70,794. None meets the stated 100,000-150,000 target. The implementation follows the explicit architectures and reports this conflict. Ask the teacher whether a resized, equal-parameter comparison is also expected; do not claim the current models satisfy that target.

Important distinction

A parameter is a stored learned number. A FLOP is a counted arithmetic operation. Latency is measured elapsed time. Accuracy is the fraction of correct predictions. A model can use few parameters but many operations.

How to run and understand the experiment

1. In PyCharm, open the existing course project that already contains lab01 and lab02. Open lab03/main.py and run that file.

2. Select the existing Python 3.11.8 interpreter listed in RUN_IN_PYCHARM.txt. Its folder is named .venv-lab2; the required Lab 3 packages are now installed there.

3. The script downloads data if missing, prints versions, profiles models, trains all three and writes results. Wait for COMPLETE. The supplied run has already finished; you can inspect it without rerunning.

4. Open report.md and results/summary.csv. The four PNG plots and three model checkpoints are in results/. verify_lab.py --results reloads the checkpoints and checks the reported scores.

Why the data is split

Training examples change the weights. Validation examples let you observe learning without updating weights. Test examples are kept for final evaluation. Here the final fifth-epoch model is tested; the test set is not used to choose an epoch or change hyperparameters.

What happens to each image

ToTensor divides each uint8 pixel by 255 and adds a channel dimension. Normalize then computes (pixel/255 - 0.2860)/0.3530. The resulting image tensor has shape (1,28,28). Batch 128 has shape (128,1,28,28).

What each model does

MLP: flatten 784 pixels; combine all inputs through dense layers. CNN: apply shared 3x3 filters to local image areas and reduce dimensions with pooling. Attention: turn each of sixteen 7x7 patches into a 64-number token; exchange information between tokens twice; average the tokens and classify.

The attention specification omits position embeddings. They were not added. Moving whole patches around leaves this model's evaluation prediction unchanged, although rearranging pixels inside a patch does not.

Your exact textbook reading

Section	Printed pages	PDF viewer pages
4.2 MLP	153-157	205-209
4.3 CNN	158-163	210-215
4.5 Attention	169-176	221-228
4.6 Architectural blocks	177-181	229-233
4.7 System-level blocks	182-188	234-240

These refer to your supplied July 31, 2025 compressed PDF. The PDF viewer page is printed page + 52. Earlier online editions may have different pages.

Paper sheet 1: definitions and MLP

Write: "These are forward-pass counts for ONE image. One MAC is one multiplication plus one addition, counted as TWO FLOPs. Biases are included in parameter counts but excluded from the representative matrix FLOPs. Activation, pooling, softmax, normalization, residual and mean-pooling operations are omitted."

For a linear layer with I inputs and O outputs:

- Parameters = I x O + O (weights plus one bias per output).

- MACs = I x O.

- FLOPs = 2 x I x O.

The 28 x 28 image has 784 pixels.

First layer, 784 -> 256:

1. Weights: 784 x 256 = 200,704.

2. Biases: 256.

3. Parameters: 200,704 + 256 = 200,960.

4. FLOPs: 2 x 784 x 256 = 2 x 200,704 = **401,408**.

Remaining layers:

Layer	Parameters	FLOPs
256 -> 128	256 x 128 + 128 = 32,896	2 x 256 x 128 = 65,536
128 -> 10	128 x 10 + 10 = 1,290	2 x 128 x 10 = 2,560

Total parameters = 200,960 + 32,896 + 1,290 = **235,146**. Total FLOPs = 401,408 + 65,536 + 2,560 = **469,504**. Flatten and ReLU add no trainable parameters.

Book: section 4.2, especially Listing 4.2, printed p. 156 / PDF page 208.

Paper sheet 2: CNN

Draw the shapes:

1 x 28 x 28 -> Conv -> 16 x 28 x 28 -> Pool -> 16 x 14 x 14 -> Conv -> 32 x 14 x 14 -> Pool -> 32 x 7 x 7 -> Flatten 1568 -> 10.

For stride s, padding p and dilation 1:

H_out = floor((H_in + 2p - K) / s) + 1.

Here K=3, p=1, s=1, so the first height is floor((28+2-3)/1)+1 = 28. Width is also 28.

Convolution parameters = K x K x C_in x C_out + C_out. Convolution FLOPs = 2 x K x K x C_in x C_out x H_out x W_out.

First convolution:

1. Products per output = 3 x 3 x 1 = 9.

2. Output values = 16 x 28 x 28 = 12,544.

3. MACs = 9 x 12,544 = 112,896.

4. FLOPs = 2 x 112,896 = **225,792**.

5. Parameters = 3 x 3 x 1 x 16 + 16 = **160**.

Second convolution (after the first pool):

1. Products per output = 3 x 3 x 16 = 144.

2. Output values = 32 x 14 x 14 = 6,272.

3. MACs = 144 x 6,272 = 903,168.

4. FLOPs = 2 x 903,168 = **1,806,336**.

5. Parameters = 3 x 3 x 16 x 32 + 32 = **4,640**.

Final linear:

- Inputs = 32 x 7 x 7 = 1,568.

- Parameters = 1,568 x 10 + 10 = **15,690**.

- FLOPs = 2 x 1,568 x 10 = **31,360**.

Totals: parameters = 160 + 4,640 + 15,690 = **20,490**; FLOPs = 225,792 + 1,806,336 + 31,360 = **2,063,488**.

Explain: weights are reused over many image positions. Few parameters do not necessarily mean few FLOPs. Max-pooling has no learned parameters, but performs comparisons; those comparisons are outside this counting convention.

Book: section 4.3, especially Listing 4.4, printed p. 161 / PDF page 213.

Paper sheet 3: attention FLOPs

Draw a 4 x 4 grid of patches on the image. Each patch is 7 x 7 pixels.

- Sequence length L = (28/7) x (28/7) = 16 patches.

- Patch input size = 7 x 7 = 49.

- Embedding dimension d = 64.

- Heads h = 4, so features per head d_head = 64/4 = 16.

- Feedforward dimension f = 128.

Patch projection:

2 x L x 49 x d = 2 x 16 x 49 x 64 = **100,352 FLOPs**.

ONE encoder layer:

Operation	Substitution	FLOPs
Q, K and V projections	2 x 16 x 64 x 3 x 64	393,216
Scores QK^T, all four heads	4 x 2 x 16 x 16 x 16	32,768
Attention weights times V	4 x 2 x 16 x 16 x 16	32,768
Output projection	2 x 16 x 64 x 64	131,072
Feedforward 64 -> 128	2 x 16 x 64 x 128	262,144
Feedforward 128 -> 64	2 x 16 x 128 x 64	262,144

For clarity: each head's Q and K has shape (16,16). QK^T is (16,16) @ (16,16), giving (16,16) scores. Four heads together cost 4 x 2 x L x L x d_head = 2 x L x L x d. Do NOT multiply by four a second time when d=64 is already used.

Attention sublayer (QKV + scores + weighted V + output): 393,216 + 32,768 + 32,768 + 131,072 = **589,824 FLOPs**.

Full encoder: 589,824 + 262,144 + 262,144 = **1,114,112 FLOPs**. Two encoders: 2 x 1,114,112 = **2,228,224 FLOPs**. Classifier after mean pooling: 2 x 64 x 10 = **1,280 FLOPs**.

Whole attention model: 100,352 + 2,228,224 + 1,280 = **2,329,856 FLOPs**.

Write: "Mean pooling turns (16,64) into (64). This is why the final classifier is applied once per image, not once per token."

Book: section 4.5.2, printed pp. 169-173 / PDF pages 221-225.

Paper sheet 4: attention parameters and profiler comparison

Patch projection: 49 x 64 + 64 = **3,200** parameters.

ONE encoder:

Component	Parameter calculation	Count
QKV	3 x (64 x 64 + 64)	12,480
Attention output projection	64 x 64 + 64	4,160
Feedforward first layer	64 x 128 + 128	8,320
Feedforward second layer	128 x 64 + 64	8,256
Two LayerNorms (scale and shift each)	2 x (64 + 64)	256

Encoder total = 12,480 + 4,160 + 8,320 + 8,256 + 256 = **33,472**. Classifier = 64 x 10 + 10 = **650**. Whole model = 3,200 + 2 x 33,472 + 650 = **70,794**.

Parameter counts do not multiply by the number of tokens: the same learned projection is reused for each token.

Copy this comparison AFTER doing the arithmetic above:

Model	Hand FLOPs	Native THOP output	2 x native	Aligned THOP MACs	2 x aligned
MLP	469,504	234,752	469,504	234,752	469,504
CNN	2,063,488	1,031,744	2,063,488	1,031,744	2,063,488
Attention	2,329,856	591,488	1,182,976	1,164,928	2,329,856

Native attention THOP includes some normalization operations but misses MultiheadAttention. Each encoder is missing 294,912 MACs, so two are missing 589,824 MACs. Four LayerNorm modules contribute 16,384 native operation units. Thus 1,164,928 - 589,824 + 16,384 = 591,488.

Write an explanation in your own words: "The default profiler has incomplete attention coverage, and its normalization rule uses a different convention. I kept its original output and then added an explicit attention rule and excluded normalization for a like-for-like comparison. Matching a custom formula is a consistency check, not independent hardware validation."

Native attention parameter count: 37,514 (incomplete). torchinfo and direct enumeration: 70,794 (complete).

For batch 128, multiply the representative FLOPs by 128: MLP **60,096,512**; CNN **264,126,464**; attention **298,221,568**. Parameter counts stay unchanged.

Paper sheet 5: measured latency, conclusion, and submission check

Read the final timing rows from report.md:

Model	Batch 1 median, 100 runs	Batch 128 median, 20 runs
MLP	0.05560 ms	0.6660 ms
CNN	0.19435 ms	5.0201 ms
Attention	0.81765 ms	5.2359 ms

These are actual recorded measurements from the supplied run, not universal constants. If you rerun the experiment, use the newly generated report values. Write: "Each timing uses one warm-up excluded from the measurement, eval mode, no autograd, CPU float32, four computation threads, and the median."

For 100 sorted times, median = (50th + 51st)/2. For 20 sorted times, median = (10th + 11th)/2. Example amortized batch-128 cost: MLP 0.6660 / 128 = 0.005203125 ms/image. This is throughput-related average service time, not single-request latency.

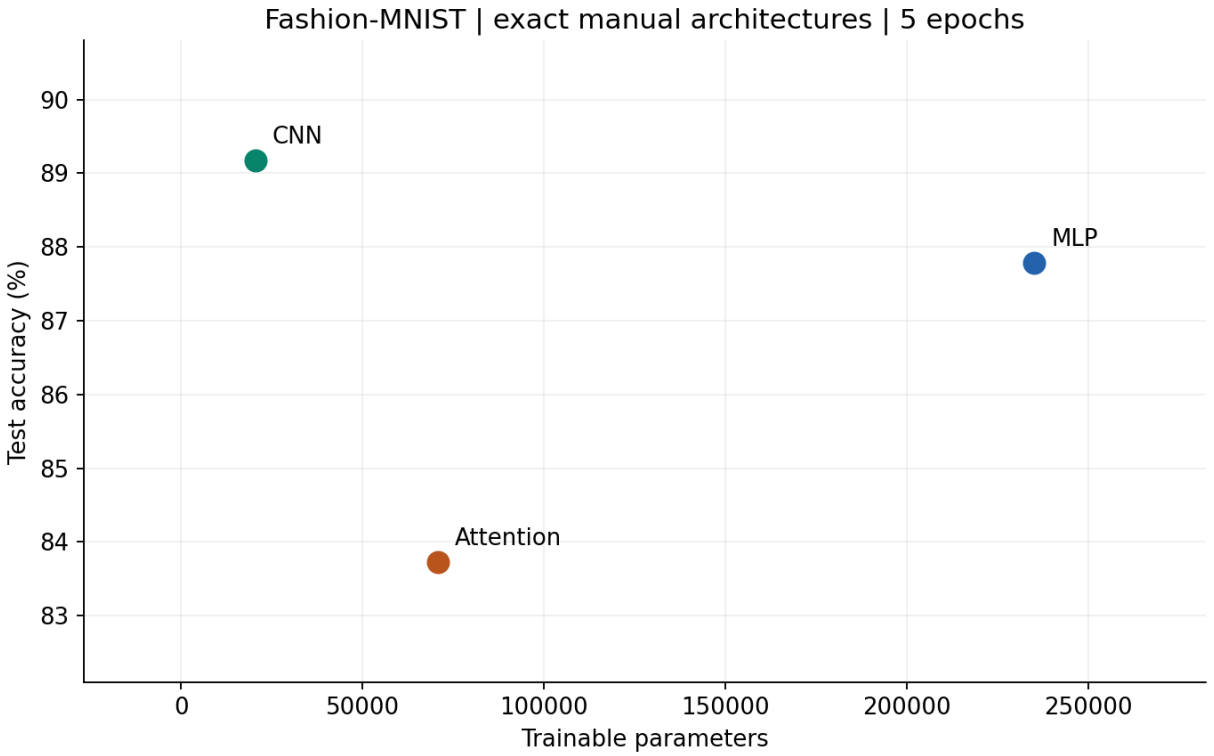
State: "The exact prescribed architectures do not satisfy the 100k-150k target. I followed the explicit layers and disclosed the contradiction."

Then explain your choice: CNN gives the best observed accuracy (89.17%) and smallest parameter count; MLP gives the lowest observed latency and 87.79% accuracy. Attention achieves 83.73% here. The best choice depends on the accuracy and resource constraint.

Final check: include your name, formulas, substituted values, units, workings, three representative layer calculations, profiler comparison, and explanation. Keep the handwritten pages with the lab; scan them into results/handwritten/.

Read the measured trade-offs

Model	Parameters	Test accuracy	B1 median ms
MLP	235,146	87.79%	0.05560
CNN	20,490	89.17%	0.19435
Attention	70,794	83.73%	0.81765



In this run, CNN achieves the highest accuracy with the fewest parameters. It is a good choice when storage and accuracy matter. MLP is faster at both tested batch sizes and gives up 1.38 percentage points of accuracy. Attention is not the best of these three after this particular five-epoch run.

Connect to the book

Table 4.2 on printed p. 181 describes MLP dense matrix work, CNN sliding windows and attention interactions. Sections 4.7.2-4.7.3 discuss memory access and broadcast, gather and reduction. CNN filter reuse explains its low weight count; MLP regular matrix operations suit optimized kernels. Attention needs several projections and intermediate tensors. These explain plausible trade-offs; they do not measure cache misses.

Do not say 'attention is always slow' or 'CNN is always best'. These are three particular models, one seed, five epochs and one CPU. Their parameter counts are unequal, and no energy or peak-memory experiment was requested.

Questions you should be ready to answer

Why count a MAC as two FLOPs?

It contains a multiplication and an addition. The manual uses this convention; the profiler commonly reports MACs.

Why does convolution reuse parameters?

The same filter is applied at every spatial location. Its arithmetic is repeated without storing new weights.

Why 16 patches?

28/7 = 4 patches along each side, so 4 x 4 = 16.

What are Q, K and V?

Learned projections: queries compare with keys to form scores, and normalized scores combine the values.

Why is attention quadratic in sequence length?

Each of L query positions compares with all L keys, giving L squared pairs.

Why is there no final softmax layer?

Cross-entropy accepts raw logits and handles the needed log-softmax. Argmax of logits gives the same predicted class.

Why eval mode and inference_mode?

eval disables dropout and chooses evaluation behavior; inference_mode avoids gradient bookkeeping. They have different jobs.

Why warm up and use a median?

Warm-up reduces first-call overhead; the median is less sensitive than a single time to occasional scheduling delays.

Why doesn't default THOP match attention?

Its installed rules miss the MultiheadAttention module but count LayerNorm work. Coverage and units differ.

What is cost-effective here?

CNN for storage and highest observed accuracy; MLP when latency matters most. Specify the deployment constraint.

Before you submit

Finish the paper workings. Check your sums. Scan the pages into lab03/results/handwritten/. Read the report and rewrite the three conclusions in your own words. Add the scans and review changes in Git, then make a new genuine commit. The current ZIP cannot contain handwriting you have not yet done.